Übung aus CHD2 - Chip-Design: Simulation

Thomas Müller-Wipperfürth, SS 2026

Homework 2: Struktur vs. Verhalten

Abgabe elektronisch bis: 15.5.2026, 23:55 Uhr

Für die Abgabe UNBEDINGT beachten:

1. Im Header jeder abzugebenden VHDL-Datei sind Name und Matrikelnummer anzugeben.
2. Die vorgefertigten VHDL-Dateien müssen verwendet werden. Außerhalb des gekennzeichneten Bereiches dürfen diese VHDL-Dateien nicht editiert werden.
3. **Abgabe über das Elearning-System**
   - Alle editierten VHDL-Dateien und die – im `sim`-Verzeichnis erzeugten – Log-Dateien sind in ein Verzeichnis `Homework2` zu legen. Zusätzlichen Dateien (z.B. *.bak oder "Erklärung.txt") werden ignoriert. Wichtig: Dateien dürfen **nicht umbenannt** werden.
   - Dieses Verzeichnis ist in ein ZIP-Archiv zu packen (**kein 7z**)! Als Name der Zip-Datei ist <Familienname>_HW2.zip zu verwenden.
4. **Ein abgegebenes Beispiel wird mit "-" bewertet, wenn**
   - der Compile-Schritt (VHDL-2002) oder die Simulation fehlschlägt,
   - die Namen der VHDL- oder Log-Dateien verändert wurden,
   - VHDL-Code außerhalb der gekennzeichneten Bereiche editiert wurden oder
   - die Log-Dateien editiert/verändert wurden.

Aufgabe 1 – 4-Bit-Addierer

1. Modelliere einen *4-Bit-Addierer* `Adder4` als *entity/architecture*. In der architecture **`Struct`** von `Adder4` ist die entity/architecture **`FullAdder(behave)`** viermal zu instanziieren und durch *signals* zu verbinden. Verwende den `FullAdder` aus Übung 3, wobei ausschließlich das Verzögerungsmodell ***transport*** verwendet werden darf. Definiere die notwendige Input- und Outputports und verwende geeignete Datentypen (z.B. `bit_vector(3 downto 0)`). Speichere das VHDL-Modell in der Datei `Adder4-ea.vhd`.
2. Teste `Adder4` in einer Testbench, die in der Datei `Adder4-tb.vhd` teilweise vorgefertigt ist. Verbinde dazu die Instanziierung des `Adder4` mit der Instanziierung der vorgegebenen Unit `Adder4Report` (Datei `Adder4Report-ea.vhd`) und einem Stimuli-Prozess `Stimul` wie in der Abbildung 1 dargestellt.
3. Es ist eine Simulation über 500 ns mit den vorgefertigten Testfällen durchzuführen. Die Testfälle dürfen nicht verändert werden.

Wichtig: In den Volladdieren muss das Verzögerungsmodell `transport` verwendet werden.

Abgabe:
`FullAdder-ea.vhd (Dateinamen nicht ändern)`,
`Adder4-ea.vhd`, `Adder4-tb.vhd`, `logHW2Bsp1.txt` und **`Adder4-ea.pdf`**

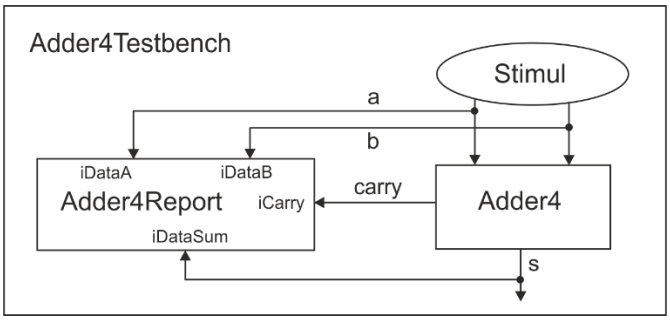

Abbildung 1: Struktur der Testbench für `Adder4`.

Aufgabe 2 – ALU (Achtung: doppelte Bewertung!)

Modelliere eine *Arithmetic Logic Unit* `ALU` als *entity* mit der *architecture* `Behave`. `ALU` besitzt zwei Inputports `iSrcA` und `iSrcB` und einen Outputport `oALUResult` für Daten vom Typ `bit_vector`. In der Testbench sind Ein- und Ausgabewerte vom Typ `bit_vector(… downto 0)` definiert. Die Breite der Input- und Output-Daten wird erst bei der Instanziierung der `ALU` festgelegt. Die Implementierung der `ALU` muss daher beliebige Datenbreiten erlauben, wobei Input- und Output-Daten immer dieselbe Breite besitzen.

Die Alu-Operation wird durch den Inputport `iALUControl` ausgewählt. Die ALU besitzt zusätzlich einen Outputport `oZero` der das Zeroflag ausgibt. Die entity Deklaration ist in der Datei `ALU-ea.vhd` vorgegeben.

`iALUControl` ist ein `bit_vector(0 to 1)`. Folgende Operationen werden durch `iALUControl` ausgewählt (`00`: Add, `01`: Sub, `10`: And, `11`: Or).

Sämtliche Daten-Outputports weisen eine Verzögerungszeit von 10 ns gegenüber den relevanten Inputports auf. Das Zero-Flag ist erst 5 ns nach Ausgabe der Daten-Outputports gültig. Es ist das *Inertial*-Modell zu verwenden.

Hinweis : Für Addition und Subtraktion dürfen die Integer-Operationen **nicht** verwendet werden. Auch Bibliotheksfunktionen auf Vektoren sind nicht zu verwenden. Schreibe eine Prozedur `addBV`, die zwei Bitvektoren addiert und als Ausgabeparameter den Ergebnisvektor liefert (die entity `FullAdder` darf nicht verwendet werden). Die Subtraktion ist durch eine Prozedur `subBV` zu realisieren.

Teste die ALU in der Testbench, die in der Datei `ALU-tb.vhd` vorgegeben ist. Simuliere dazu 3000 ns. Dabei wird im Verzeichnis `sim` die Datei `logHW2Bsp2.txt` erzeugt. Eine Abgabe der Testbench ist **nicht** erforderlich.

Abgabe:

- VHDL-Datei: `ALU-ea.vhd` und **`ALU-ea.pdf`**
- Log-Datei: `logHW2Bsp2.txt`

Wichtig: Die abgegebene *entity/architecture* von `ALU` (Datei: `ALU-ea.vhd`) wird mit der vorgegeben Testbench `ALUTestbench` (Datei: `ALU-tb.vhd`) überprüft! Die Änderung der Namen und Datentypen von *entity*, *architecture* oder *ports* von `ALU` ist unzulässig. Die vorgeschriebene Simulationszeit beträgt 3000 ns.